



IES Gabriel García Márquez

**Ciclo Formativo de Grado Superior en
Desarrollo de Aplicaciones Multiplataforma**

Curso 2022-2024

Trabajo Fin de Ciclo

SUPERCOMPARATOR

**Autor: Sergio Simón Fernández
Tutor: Christian Alcaide Moreno**

Agradecimientos

Quisiera agradecer a todo el equipo docente, del IES Gabriel García Márquez por la gran labor que han realizado durante este curso, en especial a mi tutor de proyecto Christian, por la ayuda y los consejos que me ha ofrecido durante todo el proceso.

Resumen

Se propuso desarrollar una aplicación multiplataforma para obtener información de productos en distintos supermercados y comparar sus precios. La aplicación permitiría la búsqueda simultánea de varios productos, con funciones adicionales como iniciar sesión y guardar productos.

El objetivo era crear una herramienta útil para los consumidores, ayudándoles a tomar decisiones informadas y económicas. Los usuarios podrían ingresar a la aplicación, buscar productos en diferentes supermercados y comparar precios detalladamente. También se incluiría el historial de precios y la opción de añadir productos a una lista de favoritos.

Durante el desarrollo, se identificaron desafíos significativos. Se concluyó que el desarrollo multiplataforma no era viable debido a limitaciones técnicas y de recursos, lo que llevó a enfocar el proyecto en una sola plataforma para asegurar una implementación sólida. Además, se experimentaron retrasos que impidieron la implementación de consultas múltiples.

A pesar de estos contratiempos, se lograron alcanzar otros objetivos clave. La aplicación permite a los usuarios iniciar sesión, guardar productos de interés y acceder a información detallada de precios en diferentes supermercados.

En resumen, aunque no se pudo desarrollar la funcionalidad multiplataforma ni las consultas múltiples, el proyecto avanzó significativamente en otras áreas importantes, proporcionando una herramienta eficaz para la comparación de precios y ayudando a los usuarios a ahorrar tiempo y dinero en sus compras diarias.

Palabras clave:

- Web scraping
- Multiplataforma
- API REST
- Asincronía
- Clean code
- Clean architecture
- Kotlin

- Java
- JavaScript
- MVVM (Model View, View, Model)

Índice de contenidos

Índice de figuras	XI
Índice de códigos	XIII
1. Introducción	1
1.1. Estructura del documento	1
1.2. Contexto y alcance	2
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Estado del arte	6
3.1. Súper más barato	6
3.2. Soysuper	7
4. Metodología	9
4.1. Relación con el Ciclo	9
4.2. Introducción a la Tecnología/Aplicación utilizada	10
4.2.1. Kotlin Multiplataform	10
4.2.2. Retrofit	11
4.2.3. MongoDB	11
4.2.4. Web scraping	11
4.2.5. Selenium WebDriver	12
4.2.6. Firebase	12
4.2.7. Spring Boot	14
4.2.8. Jetpack Compose	15
4.2.9. Coil	15
4.2.10. Arquitectura Hexagonal	15
4.2.11. Vico	16
4.3. Desarrollo del proyecto	17
5. Resultados y discusión	34

6. Conclusiones	36
6.1. Informe post-mortem	36
6.1.1. Lo que ha ido bien	36
6.1.2. Lo que ha ido mal	37
6.2. Trabajos futuros	39
6.3. Diagrama de Flujo	42
Bibliografía	44

Índice de figuras

3.1. Web Super Más Barato	7
3.2. Web Super Más Barato Búsqueda	7
3.3. Web SoySuper	8
4.1. Kotlin Multiplatform Schema	10
4.2. Vico Gráficos	17
4.3. Vista: Log in	19
4.4. Vista: Registro	20
4.5. Vista: Recuperación de cuenta	21
4.6. Vista: Recuperación de cuenta correo	22
4.7. Vista: Recuperación de cuenta link	22
4.8. Vista: SpashScreen	23
4.9. Vista: SpashScreen exitosa	24
4.10. Vista: SpashScreen fallo	25
4.11. Vista: Perfil-Authenticación anonima	26
4.12. Vista: Perfil-Authenticación con correo	27
4.13. Vista: Perfil-Authenticación con Google	28
4.14. Vista: Busqueda	29
4.15. Vista: Búsqueda	30
4.16. Vista: Favoritos	31
4.17. Vista: Producto Seleccionado	32
4.18. Vista: Cerrar Sesión	33
6.1. Petición a la API de Mercadona	39
6.2. Respuesta de la petición a la API de Mercadona	39
6.3. Diagrama de flujo de la aplicación	42

Índice de códigos

6.1. Ejemplo de importación de librerías en Gradle.	37
6.2. Ejemplo de importación de librerías en Version Catalog.	38

1

Introducción

En este capítulo se desarrolla una pequeña introducción a la estructura de la memoria, se contará de que va a ir cada capítulo, brevemente situando al lector de que trata cada punto y se pondrá en contexto de que trata el proyecto

1.1. Estructura del documento

La memoria del proyecto posee seis apartados diferenciados:

1 Introducción:

Se sitúa al lector proporcionando el contexto necesario para comprender el propósito y la idea central del proyecto. En esta sección se describen los puntos que se van a tratar y se explica el alcance del proyecto.

2 Objetivos:

Se redactan los hitos que se proponen alcanzar durante el desarrollo del proyecto. Se establece un objetivo general que resume la funcionalidad global del proyecto, desarrollando más a fondo las funcionalidades específicas en los objetivos específicos.

3 Estado del arte:

Se realiza una comparación de proyectos similares, analizando sus características y los criterios tomados en cuenta para el desarrollo del proyecto.

4 Metodología:

Se narra cómo se ha desarrollado el proyecto, incluyendo los conocimientos

relacionados con los módulos que han servido para su desarrollo. Se define las tecnologías usadas y se detalla el proceso de desarrollo del proyecto.

5 Resultados y discusión:

Se reflexiona sobre lo que ha supuesto el desarrollo del proyecto y se discuten las posibles mejoras.

6 Conclusiones:

Se repasan los aspectos que han funcionado en el proyecto y los que no. Se analizan los errores y aciertos durante el desarrollo del proyecto.

1.2. Contexto y alcance

En los últimos años, hemos notado un aumento considerable en el costo de vida, en especial cuando hacemos nuestras compras diarias. Al ir de compras, siempre buscamos los lugares más económicos para ahorrar dinero. Sin embargo, debido a la gran cantidad de supermercados y la diversidad de productos disponibles en la actualidad, esta tarea puede resultar tediosa y consumir mucho tiempo.

Para abordar esta problemática, se decidió desarrollar la aplicación Super-Comparator. Esta herramienta permite comparar precios de manera sencilla e intuitiva, facilitando así el ahorro de tiempo, esfuerzo y dinero. Con SuperComparator, los usuarios podrán encontrar rápidamente las mejores ofertas y tomar decisiones informadas sobre dónde realizar sus compras, optimizando sus decisiones y mejorando su experiencia de compra.

2

Objetivos

En este capítulo se desarrollan los principales objetivos del proyecto indicando cuál es la principal funcionalidad que debe realizar la aplicación y posteriormente detallando con más detenimiento los detalles más específicos que se deben realizar

2.1. Objetivo general

Desarrollar una aplicación en Android capaz de comparar precios de distintos supermercados y recopilar un historial de precios.

2.2. Objetivos específicos

- 1 Obtener productos de distintos supermercados.
- 2 Permitir registro e inicio de sesión online.
- 3 Permitir la realización de búsqueda de productos de manera individual.
- 4 Permitir de manera automatizada varias consultas de distintos productos.
- 5 Agregar productos a una lista de favoritos.
- 6 Permitir a los usuarios ver información detallada de los productos como su precio histórico o el precio por unidad.

- 7 Diseñar una interfaz atractiva.
- 8 Emplear un diseño minimalista
- 9 Diseñar una interfaz intuitiva y reconocible

3

Estado del arte

En este capítulo, exploraremos diversas herramientas disponibles en el mercado, detallando sus funcionalidades y características. Evaluaremos cuáles de estas características son comunes con nuestro proyecto y cuáles no. Este análisis permitirá comparar otras alternativas e identificar innovaciones en nuestro proyecto en relación con desarrollos previos. Además, nos servirá de inspiración para futuras versiones del proyecto, tomando lo mejor de cada herramienta.

3.1. Súper más barato

Súper más barato [4] permite elegir los supermercados en los que se desea comparar los precios de los productos. Al realizar una búsqueda, se mostrará el nombre del producto, una imagen del mismo, su precio y el precio por unidad.

Súper Más Barato es una página web, a diferencia del proyecto que está desarrollado en Android. Supercomparator ofrece una búsqueda de productos por tienda, muy similar a Súper Más Barato. Sin embargo, en Súper Más Barato no es posible elegir el filtro de precio, añadir productos a una lista de favoritos o ver su historial de precios.

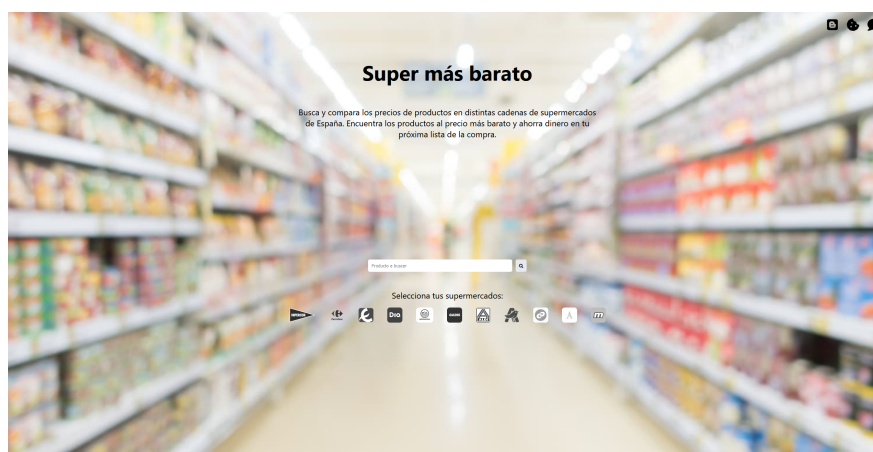


Figura 3.1: Web Super Más Barato

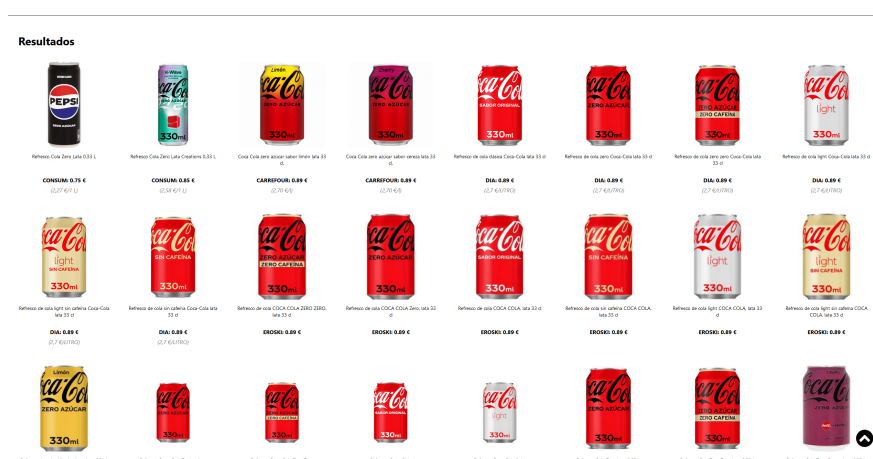


Figura 3.2: Web Super Más Barato Búsqueda

3.2. Soysuper

Soysuper [5] es una web que cuenta con aplicaciones disponibles tanto para Android como iOS. Aunque ofrece las funciones básicas de búsqueda y comparación entre supermercados, destaca por una característica innovadora: la posibilidad de agregar productos a la cesta del supermercado directamente desde su página web. Actualmente, esta función solo está disponible para productos de la misma cadena de supermercados. Otra característica notable es la presencia de una variedad de productos recomendados, sin necesidad de realizar búsquedas previas, lo que sugiere la existencia de una base de datos propia que abarca todos los productos de los supermercados, o al menos, de búsquedas anteriores.

Soysuper posee la característica única de añadir productos a la cesta de su-

permercados, funcionalidad que no está presente en el proyecto. Sin embargo, a diferencia de Soysuper, el proyecto tiene la capacidad de almacenar un historial de precios y añadir productos a una lista de favoritos.

Soysuper

🔍 Dinos, ¿qué buscas?

[Crear cuenta](#) [Entrar](#)

Tu supermercado online. Fácil, fácil.
Somos un agregador de supermercados online.
Para que hagas tu lista de la compra de forma rápida y sencilla.

[Regístrate en Soysuper](#)

[Video ¿Cómo funciona?](#)
[Descubre la fragancia de Soysuper](#)

- ✓ Escoge tu súper favorito y haz tu cesta.
- ✓ Compara precios, ofertas y plazos de entrega.
- ✓ Y el súper que elijas te llevará la compra a casa.

[Cerrar](#)

Oportunidades
Tiendas de marca

Buscar por categorías

- Aperitivos 6.302
- Bazar y Casa 23.397
- Bebés y Niños 4.323
- Bebidas 19.388
- Cafés, Cacaos e Infusiones 3.647
- Cereales y Galletas 3.700
- Chocolates y Dulces 4.813
- Congelados 5.732
- Conservas, Sopas, Aceites y Condimentos 15.119
- Dietéticos 2.308
- Droguería 10.341
- Frescos y Charcutería 30.772
- Lácteos y Huevos 6.551
- Mascotas 5.953
- Ocio y Cultura 6.486
- Panadería, Pastelería y Repostería 7.716
- Pasta, Arroz y Legumbres 2.610
- Perfumería y Parafarmacia 23.529
- Solidaridad 5

Buscar por subcategorías

Nueva cesta
Tu lista de la compra

Añade productos a tu lista

¿Te ayudamos? Tenemos + de 100.000 productos.

Hacendado Macarrón pasta 1,30€ / Paquete 1... 1,30€ / 1 kg Añadir	Hacendado Tomate frito 1,40€ / 3 x 400 g 1,17€ / 1 kg Añadir	Hacendado Atun claro en aceite vegetal girasol (abre fácil solapin) 4,20€ / Pack 6 x 5... 12,50€ / 1 kg Añadir	Hacendado Lache semidesnatada 5,01€ / 6 x 1 l 0,83€ / 1 l Añadir
Bosque Verde Servilleta papel 1 capa 30x30 cm blanca 1,24€ / Paquete 2... 0,01€ / 1 unidad Añadir	Hacendado Atún claro aceite oliva (abre fácil solapin) 5,41€ / Pack 6 lat... 15,03€ / 1 kg Añadir	Hacendado Pan molde blanco sin corteza 1,24€ / 450 g 2,76€ / 1 kg Añadir	Hacendado Tortilla de patata y cebolla 2,60€ / Bandeja 6... 4,33€ / 1 kg Añadir

Tu cesta
0 productos
Gastos de envío 0 €

- 0 productos
Gastos de envío + 7,21 €
- 0 productos
Gastos de envío + 7 €
- 0 productos
Gastos de envío + 7 €
- 0 productos
Gastos de envío + 3,99 €
- 0 productos
Gastos de envío + 6,95 €
- 0 productos
Gastos de envío + 7 €
- 0 productos
Gastos de envío + 8 €
- 0 productos
Gastos de envío + 8 €
- 0 productos
Gastos de envío + 6 €

Figura 3.3: Web SoySuper

4

Metodología

En este capítulo, se detallan las metodologías utilizadas en el proyecto, su relación con los ciclos impartidos durante el curso, sus puntos en común y cómo han contribuido al desarrollo del proyecto. Se describirán las tecnologías empleadas, su papel en el proyecto y se proporcionará una definición de cada una. Además, se narrará el proceso de desarrollo del proyecto, incluyendo las complicaciones encontradas y los detalles del desarrollo, justificando las decisiones tomadas.

4.1. Relación con el Ciclo

El proyecto está estrechamente vinculado con las materias de aplicaciones móviles, programación de procesos y acceso a datos.

La aplicación, desarrollada para Android, se fundamenta en los conocimientos adquiridos en clase. Sin embargo, se decidió utilizar Compose como alternativa a la construcción de vistas mediante XML. Además, se adoptó un enfoque orientado a la arquitectura de código, implementando el patrón MVVM (Model-View-ViewModel). Se exploraron librerías como Firebase para la autenticación, Retrofit para las solicitudes de red y Coil como evolución de Picasso para proporcionar imágenes de manera offline.

En la asignatura de programación de procesos, se aplicaron los conocimientos sobre hilos para realizar consultas asíncronas entre diferentes tiendas, reduciendo así el tiempo de espera entre las solicitudes. También se desarrolló una API

Rest con Spring Boot, investigando su funcionamiento y buscando formas más eficientes y legibles de desarrollar código, utilizando principios de Clean Code y una arquitectura hexagonal.

En la materia de acceso a datos, se emplearon los conocimientos sobre filtrado y obtención de datos de archivos JSON y XML. Además, se ampliaron estos conocimientos con el uso de bases de datos no relacionales, específicamente MongoDB.

4.2. Introducción a la Tecnología/Aplicación utilizada

A continuación se describirán las tecnologías más destacables usadas en el proyecto y que parte han desempeñado para su realización

4.2.1. Kotlin Multiplatform

En la página oficial de Kotlin [11], se define a Kotlin Multiplatform como una tecnología diseñada para reducir el tiempo en proyectos multiplataforma. Esto se logra disminuyendo el tiempo dedicado a escribir y mantener el mismo código para diferentes plataformas, al tiempo que se conserva la flexibilidad y los beneficios de la programación nativa.

Kotlin Multiplatform permite mantener una única base de código para la lógica de la aplicación en diversas plataformas. Además, ofrece las ventajas de la programación nativa, incluido un excelente rendimiento y acceso completo a los SDK de cada plataforma.

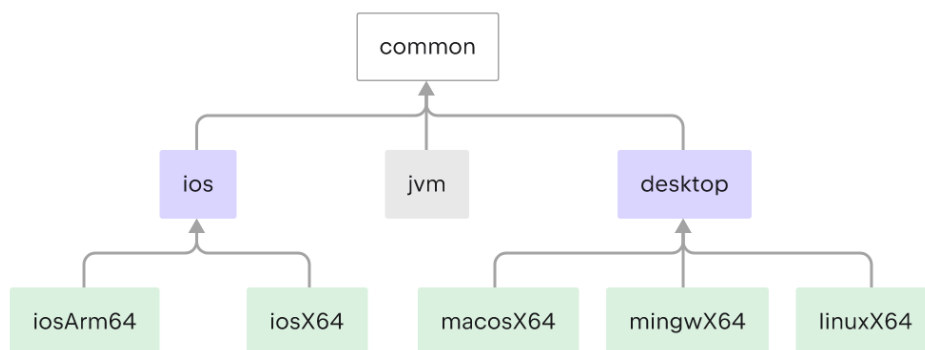


Figura 4.1: Kotlin Multiplatform Schema

4.2.2. Retrofit

Retrofit [12] es una biblioteca para Android y Java desarrollada por Square que facilita la creación de clientes HTTP para consumir APIs RESTful. Simplifica las tareas de realizar solicitudes HTTP, como GET, POST, PUT y DELETE, al convertir automáticamente las respuestas JSON o XML en objetos Java o Kotlin, utilizando convertidores específicos.

4.2.3. MongoDB

MongoDB [13] es una base de datos NoSQL orientada a documentos, desarrollada por MongoDB Inc., que se utiliza para almacenar y gestionar grandes volúmenes de datos de manera flexible y escalable. A diferencia de las bases de datos relacionales tradicionales que almacenan datos en tablas con filas y columnas, MongoDB almacena datos en documentos BSON (una extensión binaria de JSON), permitiendo una estructura de datos más dinámica y ágil.

4.2.4. Web scraping

Para el proyecto Supercomparator se ha sido necesario el uso **web scraping**, según la página web Geeksforgeeks define WebScraping [2] de la siguiente manera:

- Imaginemos que necesitas obtener información de un sitio web. Copiar y pegar puede servir para pequeñas cantidades, pero ¿qué hacer si requieres grandes cantidades de datos de manera rápida? En ese caso, copiar y pegar simplemente no es una opción viable. Ahí es donde entra en juego el Web Scraping. A diferencia del tedioso proceso manual de recopilación de datos, el Web Scraping emplea técnicas automatizadas para extraer miles o incluso millones de conjuntos de datos en un lapso de tiempo mucho más corto.

En el proyecto se han utilizado dos metodologías de web scraping:

- Llamado de una api del comercio

Cuando se solicitan datos al api del comercio, estos llegan en formato json, describiendo los productos consultados. A partir de esta información, se realiza una conversión a un objeto Kotlin. Este objeto almacena la información completa o parcial del producto, dependiendo del alcance de la consulta y la relevancia de los datos obtenidos.

- Filtrado del html del comercio

El proceso de filtrado mediante html es bastante similar. Se realiza una consulta que devuelve todo el contenido de la página web del comercio. A continuación, se trabaja con este html para extraer los datos relevantes y almacenarlos en un objeto Kotlin. Este filtrado se lleva a cabo utilizando las etiquetas html, asociadas a clases css, que permiten distinguir los distintos campos de la página y extraer los datos necesarios.

4.2.5. Selenium WebDriver

Selenium WebDriver, según la página web *El blog de python* [3] es una herramienta de automatización de pruebas que permite interactuar con aplicaciones web de forma automatizada.

La elección de Selenium WebDriver se fundamenta en su funcionamiento interno. Esta herramienta opera mediante un controlador de navegador, como Firefox, para ejecutar las pruebas de forma automática. Crea un navegador virtual que permite ejecutar el JavaScript necesario en aquellas páginas que requieren este tipo de interacción.

Este enfoque ha permitido realizar consultas a las páginas web de Hipercor y Mercadona. En el caso de Hipercor, su API siempre devolvía un error 404 al utilizar Retrofit (véase 4.2.2). Con Mercadona, el problema surgía porque, al hacer una llamada HTTP simple para obtener su HTML, la respuesta incluía un error indicando que no se pudo ejecutar el código JavaScript. Para solucionar esto, se ha utilizado Selenium para emular un navegador y hacer scraping de su contenido en HTML (véase la sección 4.2.4).

4.2.6. Firebase

Firebase [6] es una plataforma para el desarrollo de aplicaciones web y móviles que permite a los desarrolladores ahorrar tiempo en la programación. Ofrece soluciones multiplataforma (iOS, Android, Unity, etc.) y cuenta con la infraestructura de Google, lo que permite escalar automáticamente a medida que la aplicación crece. Además, Firebase permite medir el rendimiento del proyecto mediante Firebase Analytics.

Entre sus herramientas se encuentra:

- **Firebase Cloud Messaging**

Firebase Cloud Messaging (FCM) es un servicio que permite enviar notificaciones y mensajes a aplicaciones móviles y web sin costo alguno. FCM soporta distintos tipos de mensajes, incluyendo notificaciones que pueden

mostrarse al usuario y datos que la aplicación puede manejar de forma silenciosa en el fondo. Es ideal para alertar a los usuarios sobre nuevos eventos, actualizaciones de contenido, mensajes personales, y más, con opciones de envío segmentado basadas en temas, grupos de dispositivos o usuarios individuales.

- **Firebase Auth**

Firebase Authentication proporciona una forma rápida y segura de manejar la autenticación de usuarios en aplicaciones. Admite varios métodos de autenticación, incluyendo correo electrónico y contraseña, proveedores de identidad federada como Google, Facebook y Twitter, así como métodos anónimos. Firebase Auth facilita la implementación de flujos de inicio de sesión y registro, y se integra con otros servicios de Firebase para brindar una experiencia de usuario unificada y segura.

- **Firebase Database**

Firebase Realtime Database es una base de datos NoSQL alojada en la nube que permite almacenar y sincronizar datos entre usuarios en tiempo real. Todos los datos se almacenan en formato JSON y se sincronizan en tiempo real a cada cliente conectado. Es ideal para aplicaciones colaborativas como chats, juegos en línea y aplicaciones que requieren datos en tiempo real. Ofrece funcionalidades como el almacenamiento sin conexión, lo que permite que las aplicaciones sigan funcionando y se sincronicen cuando la conexión se restablezca.

- **Firebase Storage**

Firebase Storage ofrece una solución robusta y escalable para almacenar y servir archivos generados por usuarios, como fotos, videos y otros contenidos multimedia. Está construido sobre Google Cloud Storage, garantizando una alta disponibilidad y escalabilidad. Firebase Storage maneja de forma segura las subidas y descargas de archivos, y se integra fácilmente con Firebase Authentication para permitir el control de acceso basado en los usuarios. Proporciona capacidades de reanudación de cargas y descargas, lo que mejora la experiencia del usuario en redes inestables.

- **Firebase Cloud Firestore**

Firebase Cloud Firestore es una base de datos flexible y escalable para desarrollo móvil, web y servidor. Es una base de datos NoSQL que permite el almacenamiento, la sincronización y la consulta de datos de aplicaciones a una escala global. Firestore ofrece consultas sofisticadas, estructura de datos en tiempo real y soporte para transacciones. A diferencia de Realtime Database, Firestore tiene una estructura jerárquica y soporta colecciones y documentos anidados, lo que facilita la organización y consulta de datos complejos. También se sincroniza en tiempo real y ofrece capacidades sin conexión, asegurando que las aplicaciones funcionen de manera eficiente en

cualquier circunstancia de conectividad.

Cada uno de estos módulos proporciona herramientas esenciales para desarrollar aplicaciones robustas y eficientes, permitiendo a los desarrolladores centrarse en la lógica de negocio sin preocuparse por la infraestructura subyacente.

Se han utilizado principalmente las funcionalidades de autenticación que proporciona Firebase, incluyendo correo/contraseña, usuario anónimo y cuenta de Google. Además, se han implementado las analíticas de Firebase para recopilar información sobre el uso de la aplicación por parte de los usuarios.

4.2.7. Spring Boot

Spring Boot [7] [8] es una extensión del popular marco de trabajo empresarial Java Spring Framework. Se destaca por su capacidad para simplificar y acelerar el desarrollo de microservicios y aplicaciones web. Spring Boot se distingue por tres características principales:

1. Configuración automática: Spring Boot ofrece una configuración automática inteligente que detecta las dependencias en el proyecto y configura automáticamente la aplicación en consecuencia. Esto reduce significativamente la cantidad de configuración manual necesaria para poner en marcha una aplicación.

2. Enfoque de configuración obstinado: Spring Boot promueve un enfoque de configuración obstinado, lo que significa que proporciona valores predeterminados sensatos para la configuración, pero permite a los desarrolladores anular estos valores fácilmente cuando sea necesario. Esto proporciona flexibilidad para adaptar la configuración según los requisitos específicos del proyecto.

3. Capacidad de crear aplicaciones autónomas: Una de las características más destacadas de Spring Boot es su capacidad para generar aplicaciones autónomas. Esto significa que una aplicación Spring Boot puede empaquetar todo lo que necesita para ejecutarse, incluida la máquina virtual Java (JVM), lo que la hace independiente de la instalación de software adicional en el entorno de ejecución. Esta característica simplifica enormemente la implementación y el despliegue de aplicaciones.

En conjunto, estas características hacen de Spring Boot una herramienta poderosa para el desarrollo rápido y eficiente de aplicaciones Java, permitiendo a los desarrolladores enfocarse en la lógica de la aplicación en lugar de preocuparse por la configuración y el entorno de ejecución.

4.2.8. Jetpack Compose

Jetpack Compose es un moderno kit de herramientas de UI (Interfaz de Usuario) para la construcción de interfaces de aplicaciones de Android de manera declarativa y con un enfoque basado en componentes. Desarrollado por Google, se integra con el ecosistema de Jetpack de Android y está diseñado para simplificar y acelerar el desarrollo de aplicaciones Android mediante la adopción de patrones de diseño más intuitivos y eficientes.

En lugar de utilizar XML para definir la interfaz de usuario, Jetpack Compose utiliza Kotlin, lo que permite a los desarrolladores describir la interfaz de usuario de manera más concisa y dinámica mediante funciones y composiciones de componentes. Esto facilita la creación de interfaces de usuario altamente personalizadas y adaptables, además de simplificar tareas como la gestión del estado de la interfaz de usuario y la interacción con los datos.

Jetpack Compose ofrece una experiencia de desarrollo más fluida al proporcionar herramientas de previsualización en tiempo real, lo que permite a los desarrolladores ver instantáneamente cómo se verán y se comportarán los cambios en la interfaz de usuario mientras escriben código. Además, está diseñado para ser compatible con las herramientas y bibliotecas existentes de Android, lo que facilita la integración en proyectos de Android existentes y la adopción gradual en nuevas aplicaciones.

La interfaz de la aplicación está realizada enteramente con Jetpack Compose, utilizando los componentes de Material 3 para darle un aspecto moderno y actual. La interfaz ha sido diseñada teniendo en cuenta el modo oscuro de los teléfonos, adaptándose automáticamente al tema del sistema. Además, en dispositivos con Android 13, la aplicación soporta los temas dinámicos del sistema.

4.2.9. Coil

Coil es una librería para la carga y manipulación de imágenes en aplicaciones Android. Permite almacenar en la caché del sistema las imágenes consultadas en internet, facilitando así su carga de manera offline.

4.2.10. Arquitectura Hexagonal

La arquitectura hexagonal^[10] es un modelo de diseño de aplicaciones de software en torno a la lógica de dominio para aislarla de factores externos.

La lógica del dominio se especifica en un núcleo de negocio, al que llamaremos la parte interna, mientras que el resto son partes externas. El acceso a la lógica

del dominio desde el exterior está disponible a través de puertos y adaptadores.

La arquitectura hexagonal define la interna y exterior. Se dividen en tres capas:

- Aplicación (Exterior)
- Dominio (Interior)
- Infraestructura (Exterior)

A través de la capa de aplicación, el usuario u otros programas interactúan con la aplicación. Esta capa debe contener elementos como interfaces de usuario, controladores RESTful y bibliotecas de serialización JSON. Incluye todo lo necesario para exponer el acceso a la aplicación y organizar la ejecución de la lógica del dominio.

En la capa de dominio, se mantiene el código que implementa la lógica empresarial. Esta es el núcleo de la aplicación. Esta capa debe estar aislada tanto de la capa de aplicación como de la capa de infraestructura. Además, debe contener interfaces que definan la API para comunicarse con partes externas, como la base de datos, con la que interactúa el dominio.

Finalmente, la capa de infraestructura contiene todo lo que la aplicación necesita para funcionar, como la configuración de la base de datos o la configuración de Spring. También implementa las interfaces dependientes de la infraestructura definidas en la capa de dominio.

Este enfoque permite separar el trabajo en cada capa. Podemos enfocarnos en una capa sin afectar a las demás.

Además, estas capas son naturalmente más fáciles de entender por qué cada una se centra en su propia lógica.

Otra gran ventaja es que la lógica del dominio está aislada de todo lo demás. La parte del dominio solo contiene la lógica empresarial y se puede mover fácilmente a un entorno diferente.

4.2.11. Vico

La librería Vico para Android es una biblioteca gráfica de código abierto diseñada para facilitar la creación de gráficos y visualizaciones de datos en aplicaciones Android. Desarrollada por Javieresc, Vico proporciona una API intuitiva y flexible que permite a los desarrolladores integrar diversos tipos de gráficos, como gráficos de líneas, gráficos de barras, y otros tipos de visualizaciones, de manera rápida y eficiente.

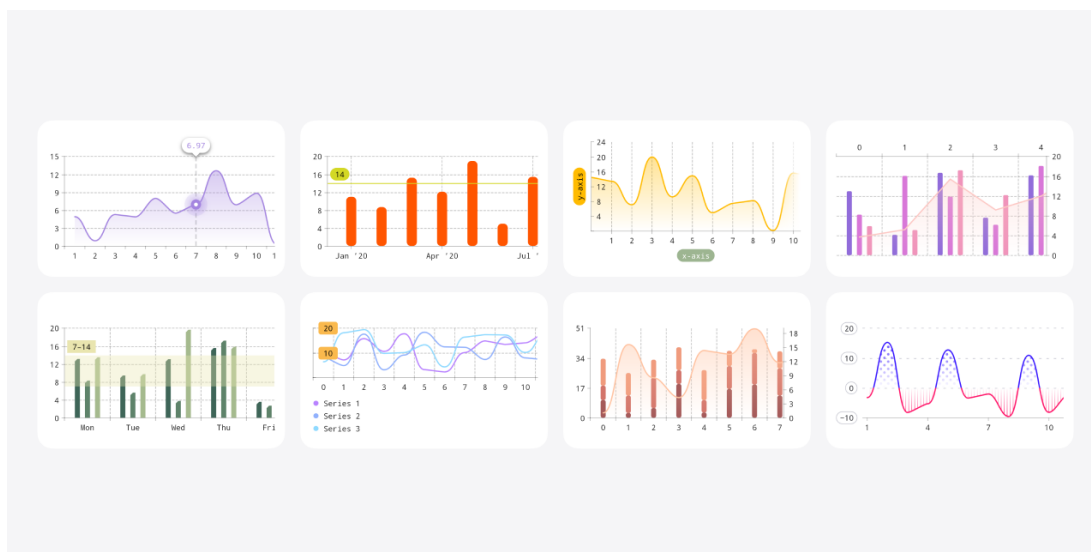


Figura 4.2: Vico Gráficos

4.3. Desarrollo del proyecto

Para el desarrollo del proyecto, inicialmente se consideró utilizar Kotlin Multiplatform (véase 4.2.1). Sin embargo, como se detalla en 6.1.2, esto resultó ser un gran desafío. La dificultad de encontrar bibliotecas compatibles, configurarlas y buscar equivalentes multiplataforma para muchos elementos de Compose retrasó significativamente el desarrollo. Debido a estos problemas, se decidió migrar el proyecto a Android usando Compose, ya que en Android sí existe una documentación detallada.

Una vez migrado el código a Android, que inicialmente solo incluía las vistas de inicio de sesión y registro, el desarrollo del proyecto avanzó más rápidamente. Durante esta fase, se investigó y ajustó la interfaz para desarrollar un estilo propio en la app y se exploraron métodos para implementar nuevos elementos. Además, se investigaron diversas formas de hacer scraping para extraer la información de las tiendas (véase 4.2.4).

No había una manera estándar para acceder a la información, como se detalla en 6.1.2, lo que resultó en que cada comercio tuviera sus propias particularidades e inconvenientes. Por ejemplo, Mercadona no tenía una API pública y no se podía acceder de manera estándar con simples peticiones HTTP utilizando alguna librería como Retrofit (véase 4.2.2). Para solucionar esto, se usó la librería Selenium (véase 4.2.5), que emula un navegador para automatizar tareas y puede ejecutar JavaScript, lo cual es necesario para poder hacer peticiones a la tienda de Mercadona.

El inconveniente fue que Selenium no se puede ejecutar de manera nativa en Android, ya que necesita un driver de navegación y una librería adaptada para Android. Por esta razón, se optó por desarrollar una API en Spring Boot (véase 4.2.7) para manejar estas solicitudes.

La API en Spring Boot se desarrolló teniendo en cuenta la arquitectura hexagonal (véase 4.2.10), la cual ofrece grandes ventajas. La principal razón para optar por esta arquitectura fue la facilidad que proporciona para mantener el código y, aún más importante, la capacidad de escalarlo con facilidad. Actualmente, la API utiliza una base de datos en MongoDB (véase 4.2.3), pero si en el futuro se decide migrar la información a otro tipo de base de datos, como MySQL o Redis, esto se podrá hacer con un mínimo de modificaciones en el código, manteniendo la misma lógica o lo más similar posible según lo permita la tecnología.

Además, la arquitectura hexagonal facilita la adición de nuevas funcionalidades o casos de uso, permitiendo su integración sin que afecten a los demás casos de uso, manteniéndolos independientes unos de otros.

En Android se optó por una arquitectura orientada al MVVM (model view viewmodel) en el cual la vista contiene solo contiene los elementos de esta, el view model contiene la lógica relacionada con la vista y el modelo contiene los módulos generales que se usaran en la aplicación como puede ser la base de datos o el cliente HTTP.

Se ha implementado la posibilidad de autenticarse en la aplicación mediante Firebase (véase 4.2.6), utilizando su característica de autenticación. Actualmente, se pueden acceder a la aplicación de tres maneras: mediante correo/contraseña, cuenta de Google y de manera anónima, como se puede observar en la figura 4.10.

En la pantalla de inicio de sesión, se encuentran los campos de Email y Password, destinados a iniciar sesión con el método de correo/contraseña. Además, más abajo, se presenta la opción de iniciar sesión de manera anónima, con el texto "No Account", y la opción de iniciar sesión con una cuenta de Google.

Si nos fijamos debajo de los botones para iniciar sesión de manera anónima y con una cuenta de Google, se puede ver el texto "Don't have an account? **Create now**". Al seleccionar "**Create now**", seremos redirigidos a la pantalla de registro de cuentas de correo/contraseña.

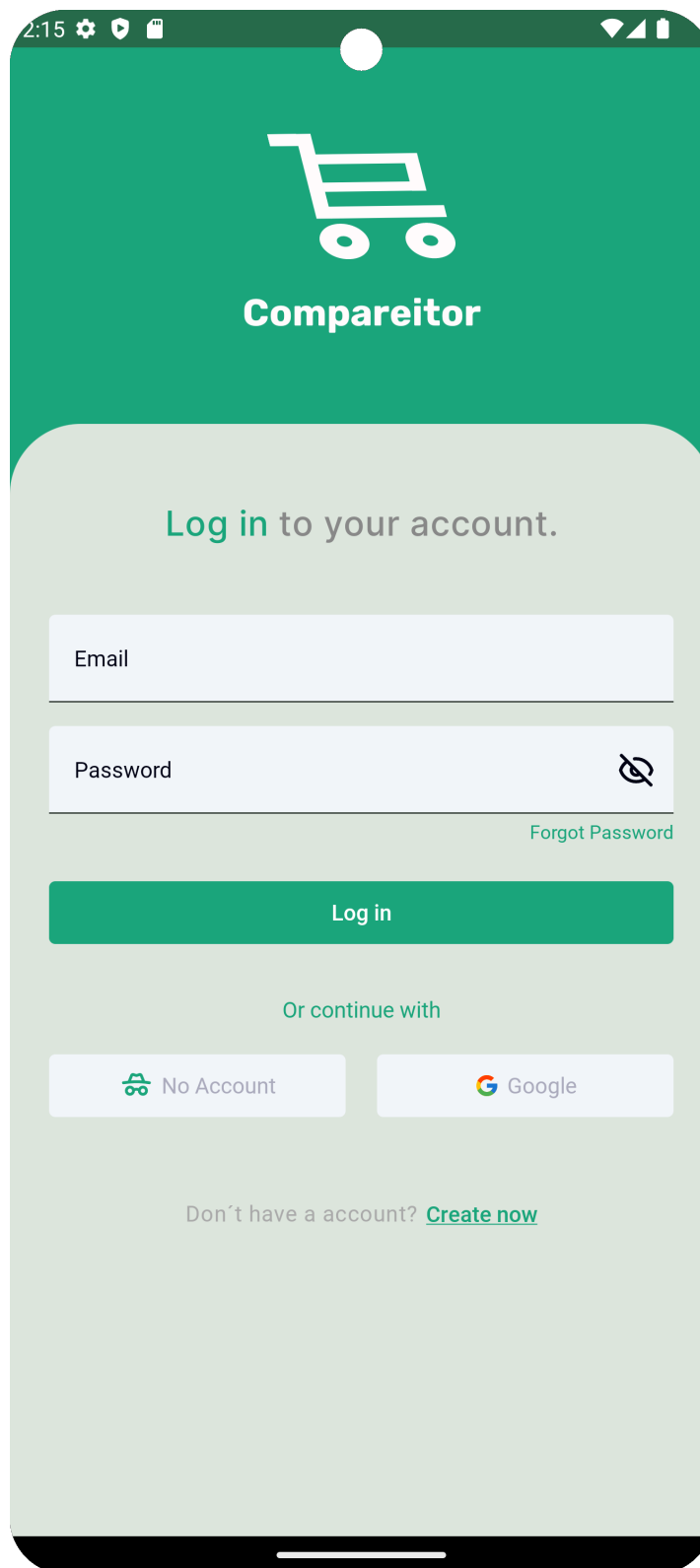


Figura 4.3: Vista: Log in

En la pantalla de registro, observamos una disposición similar a la de la pantalla de inicio de sesión, con la adición de un campo extra para la confirmación de contraseña. Esta adición se realiza para asegurar que el usuario no cometa errores al ingresar la contraseña. Además, los botones de inicio de sesión alternativos desaparecen en esta pantalla. Al igual que en la pantalla de inicio de sesión 4.10 al pulsar en el texto resaltado **"Sign in"** iremos a la pantalla de inicio de sesión

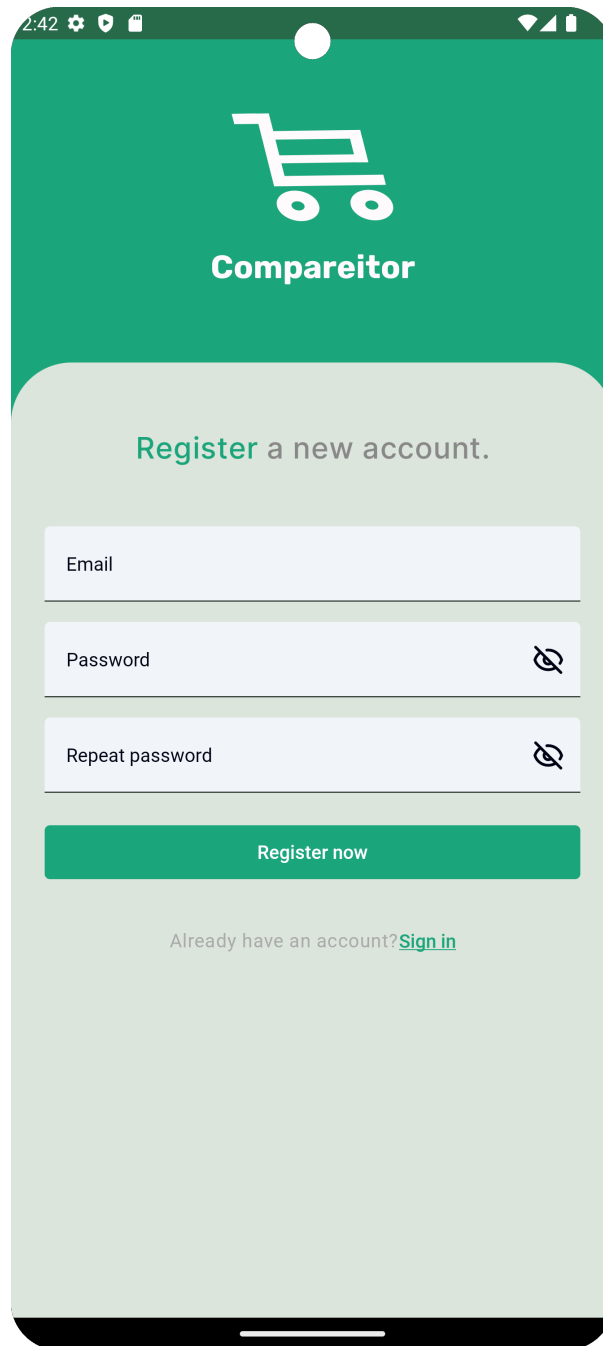


Figura 4.4: Vista: Registro

En la pantalla de inicio de sesión (véase 4.10), debajo del campo de contraseña, había un pequeño texto que decía **”Forgot Password”**. Al seleccionar esta opción, se nos lleva a una pantalla para recuperar la cuenta. Si hemos iniciado sesión con el método de correo/contraseña, se enviará un correo para recuperar la cuenta a la dirección que indiquemos.

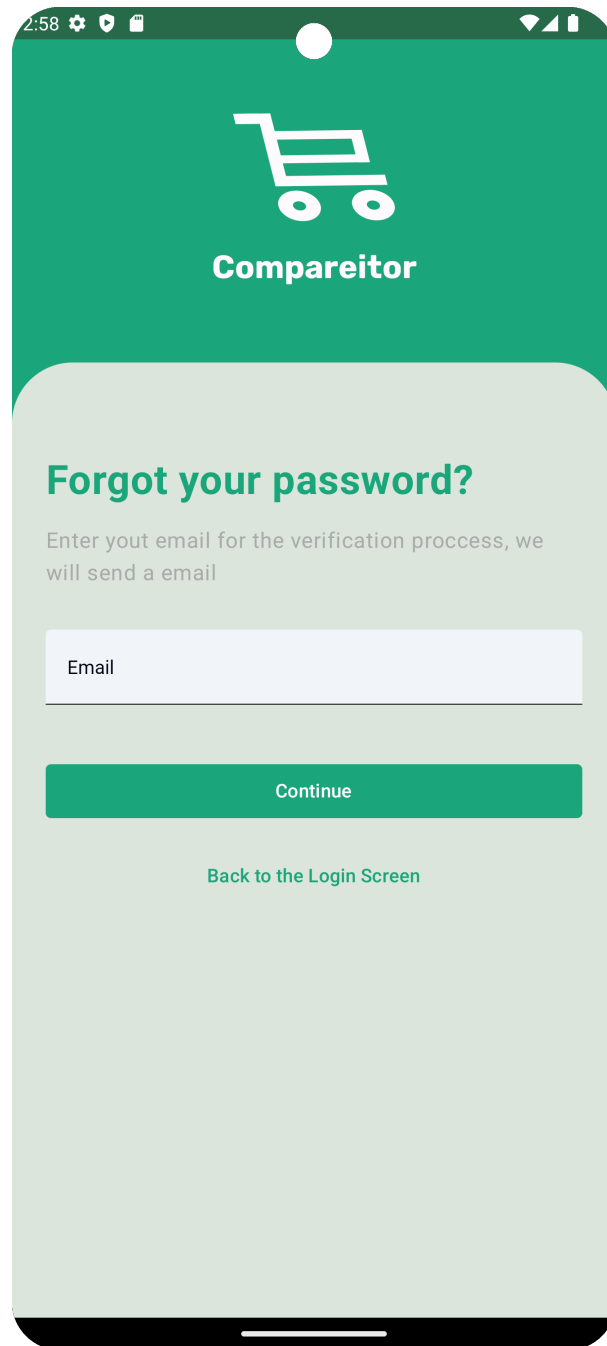


Figura 4.5: Vista: Recuperación de cuenta

Al escribir el email nos llegará un correo como el siguiente:

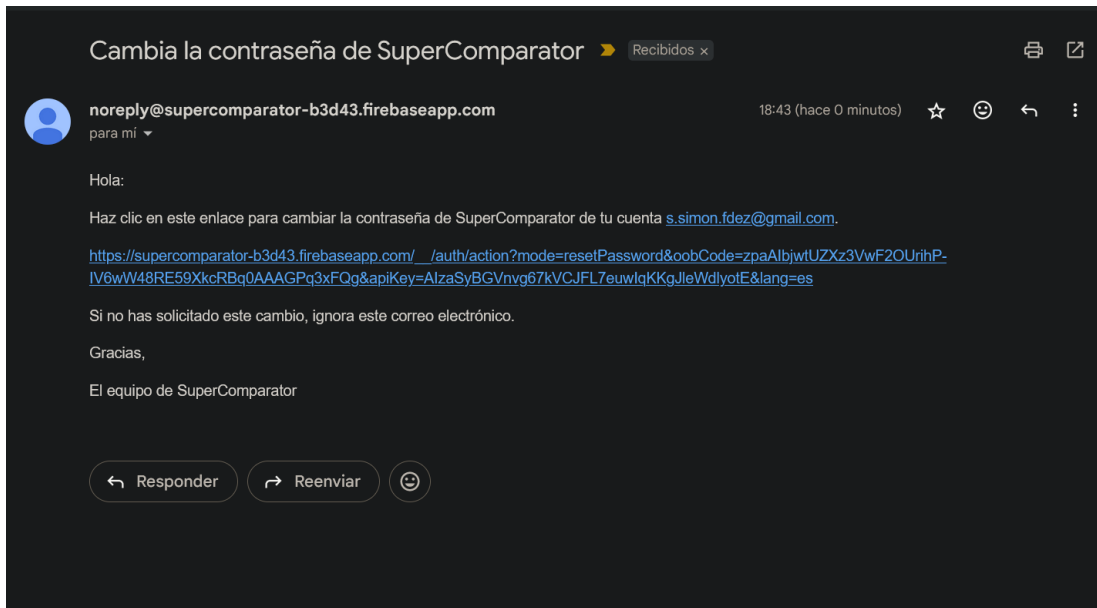


Figura 4.6: Vista: Recuperación de cuenta correo

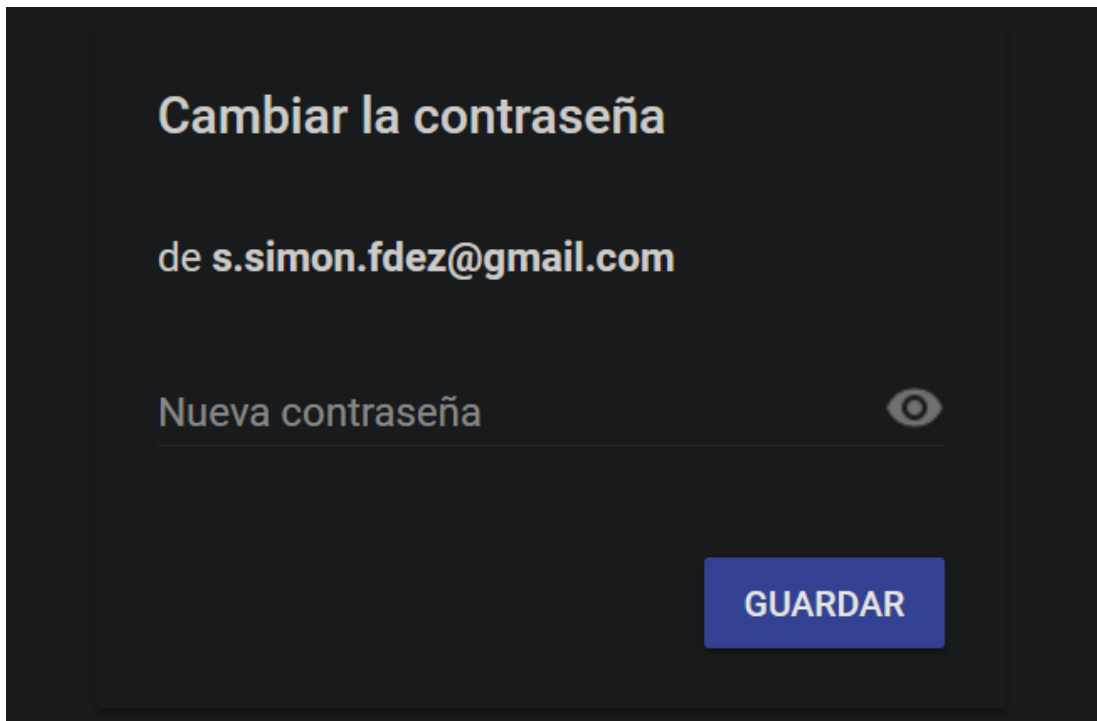


Figura 4.7: Vista: Recuperación de cuenta link

Al iniciar la aplicación, siempre se mostrará la pantalla de "Splash Screen". Esta pantalla actúa como un punto de espera mientras la aplicación se carga. En esta aplicación, la "Splash Screen" verifica que exista una conexión con la API para poder utilizar sus funcionalidades, tales como la búsqueda en diversas tiendas y la persistencia y recuperación de precios históricos de los productos buscados.

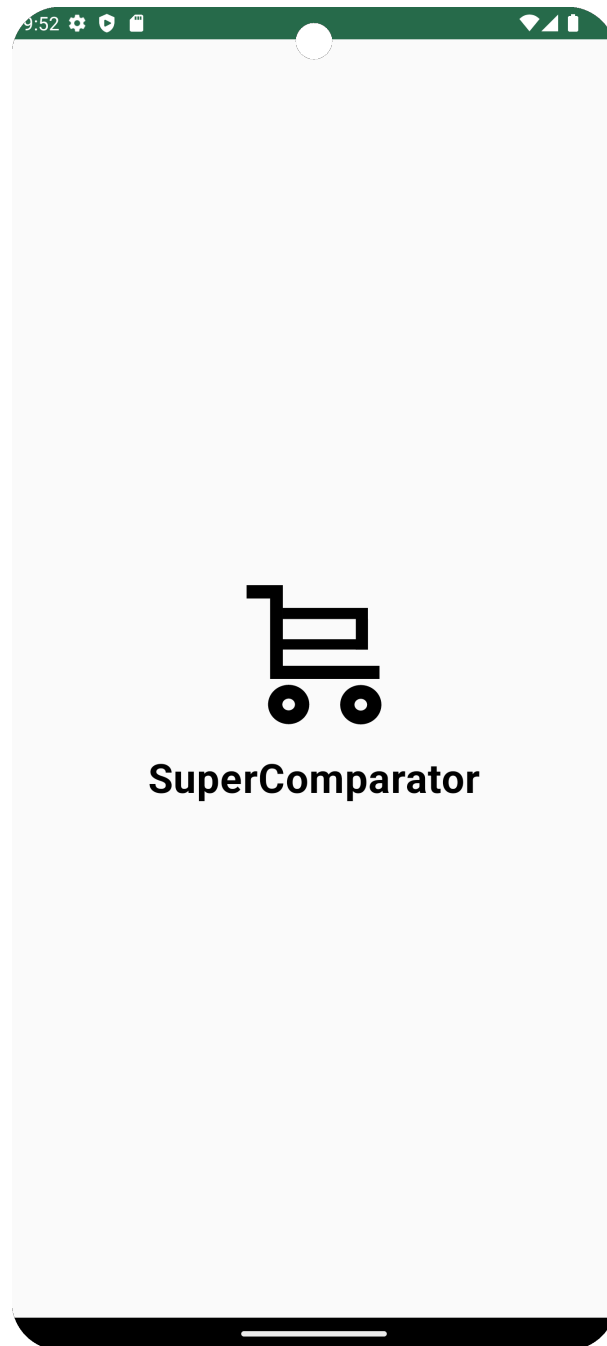


Figura 4.8: Vista: SpashScreen

Conexión exitosa

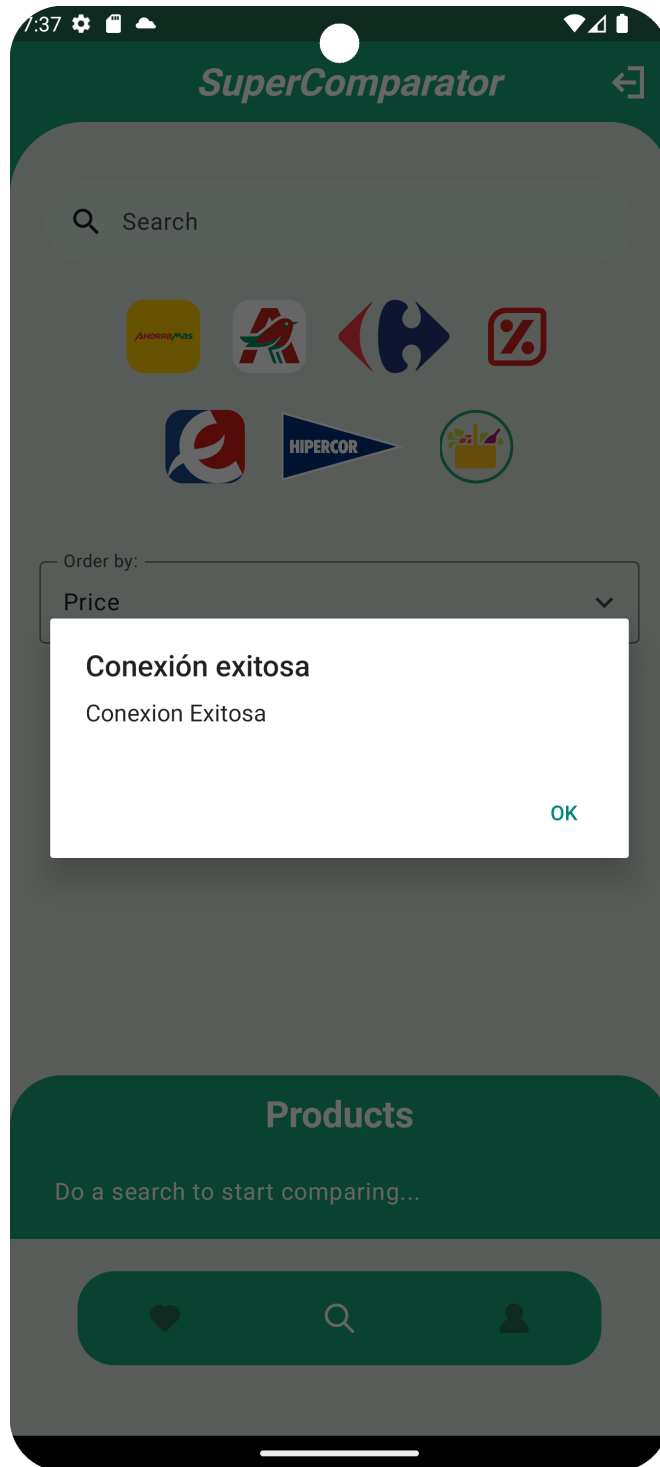


Figura 4.9: Vista: SplashScreen exitosa

Fallo en la conexión

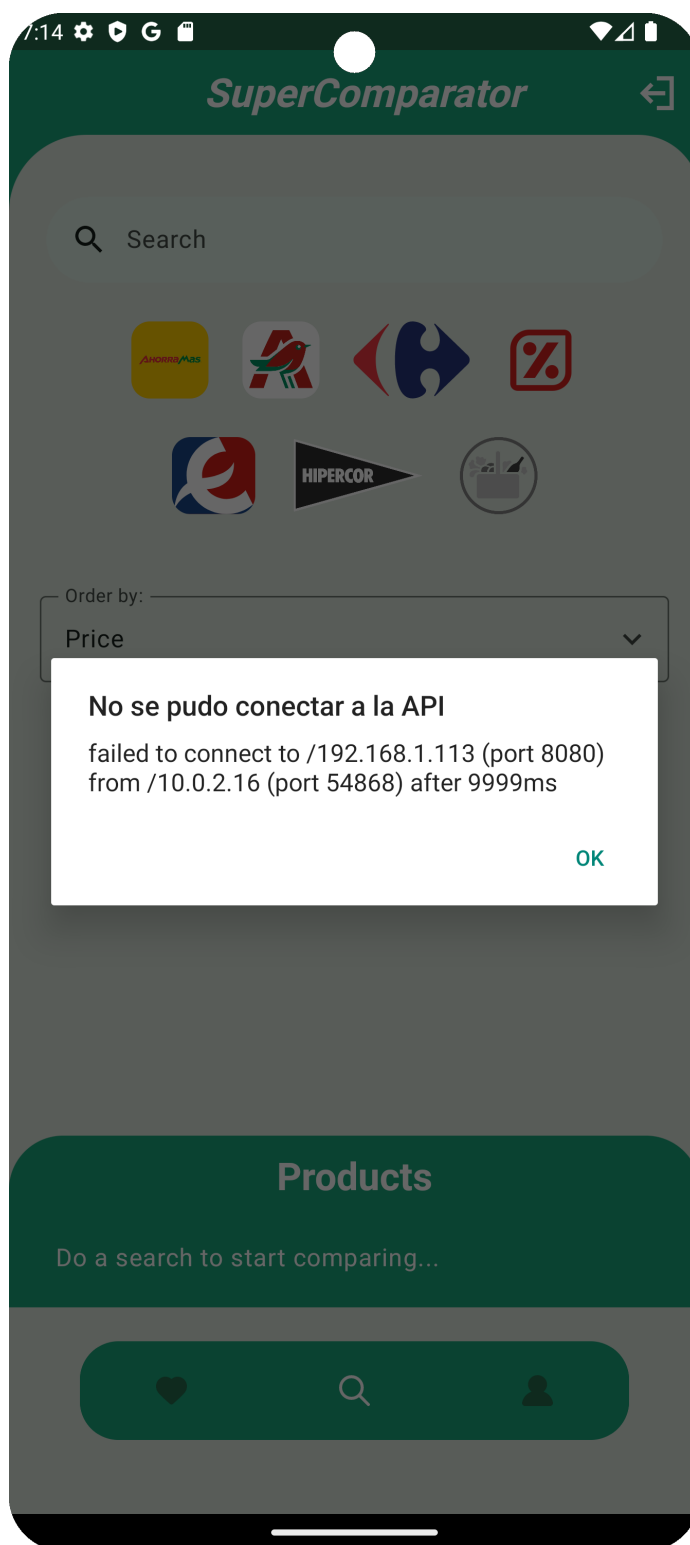


Figura 4.10: Vista: SpashScreen fallo

Al iniciar sesión dependiendo de cuál haya sido el método escogido, podremos ver en el apartado de usuario de tres maneras distintas:

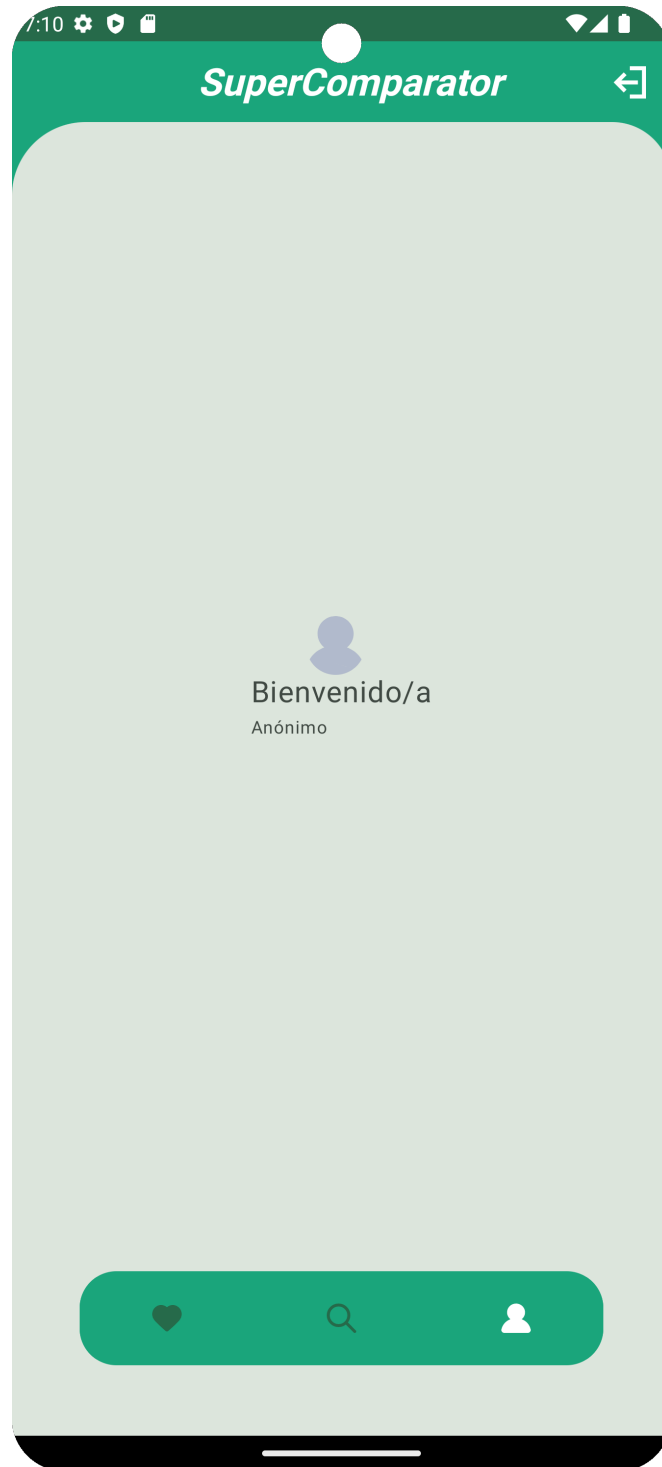


Figura 4.11: Vista: Perfil-Autenticación anonima

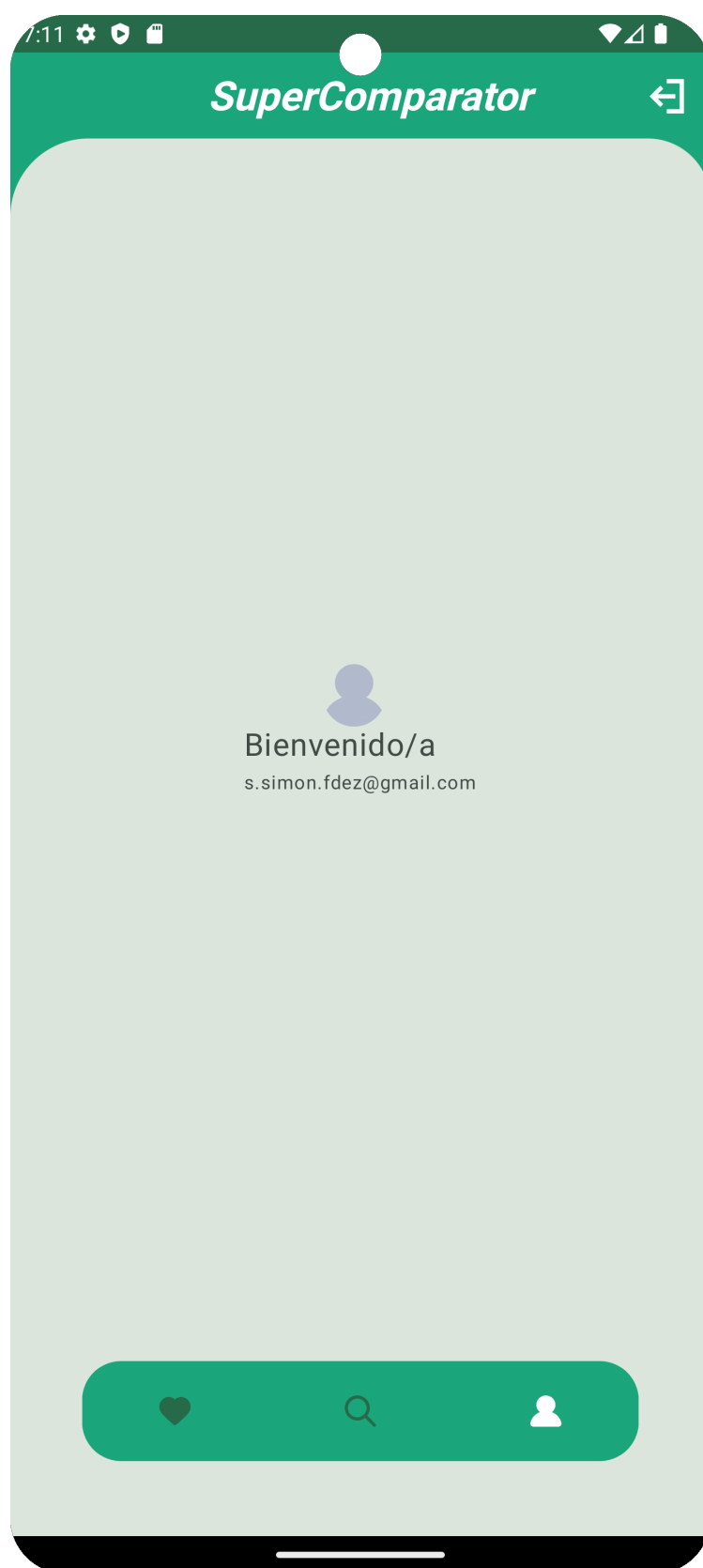


Figura 4.12: Vista: Perfil-Authenticación con correo

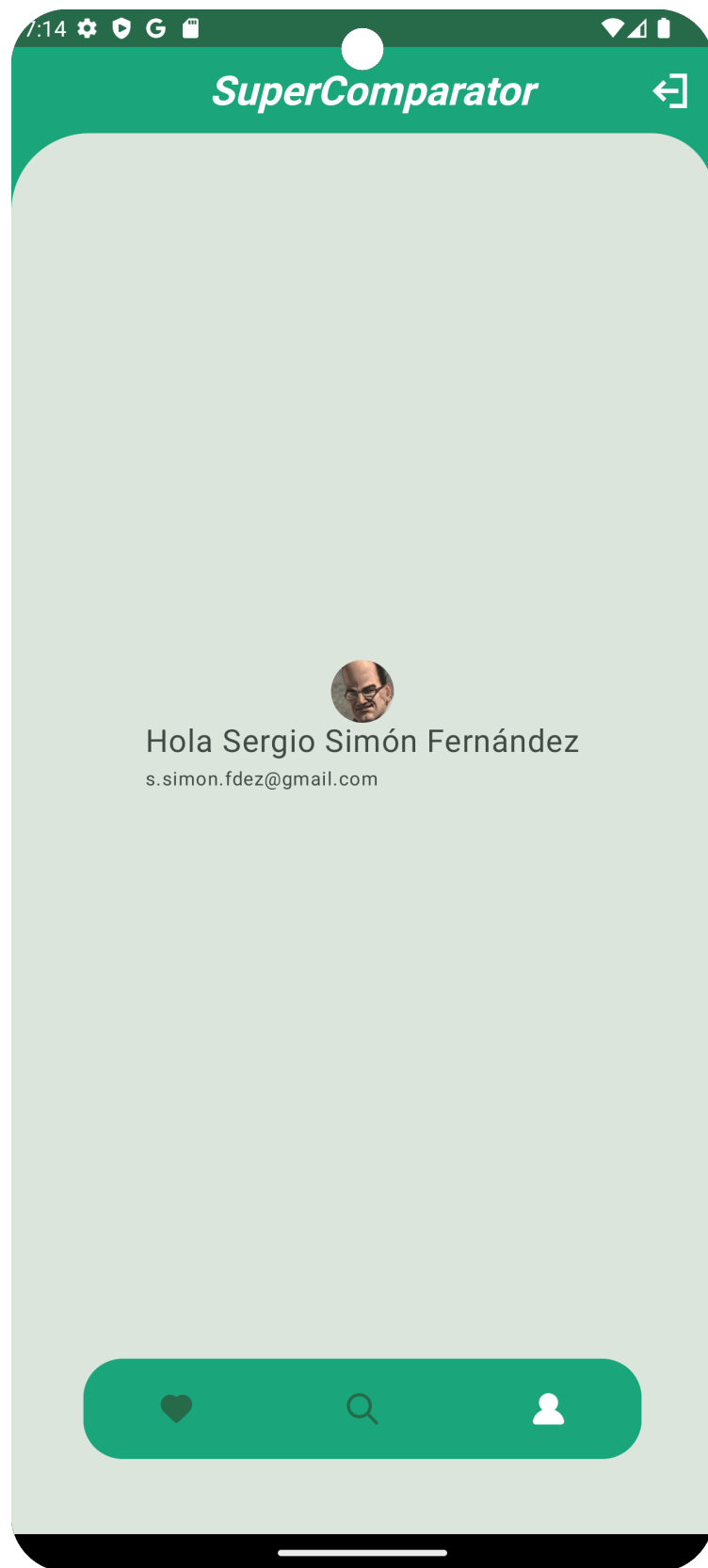


Figura 4.13: Vista: Perfil-Authenticación con Google

La funcionalidad principal del programa, que es la búsqueda de productos, se encuentra en la sección de búsqueda. En esta sección, se puede ver un área para realizar las búsquedas, con las tiendas activadas por defecto, representadas con su logo en color. También incluye un selector (spinner) para elegir el modo de filtro de productos, que puede ser por precio o precio por cantidad. Al principio, la sección de resultados de búsqueda estará vacía.

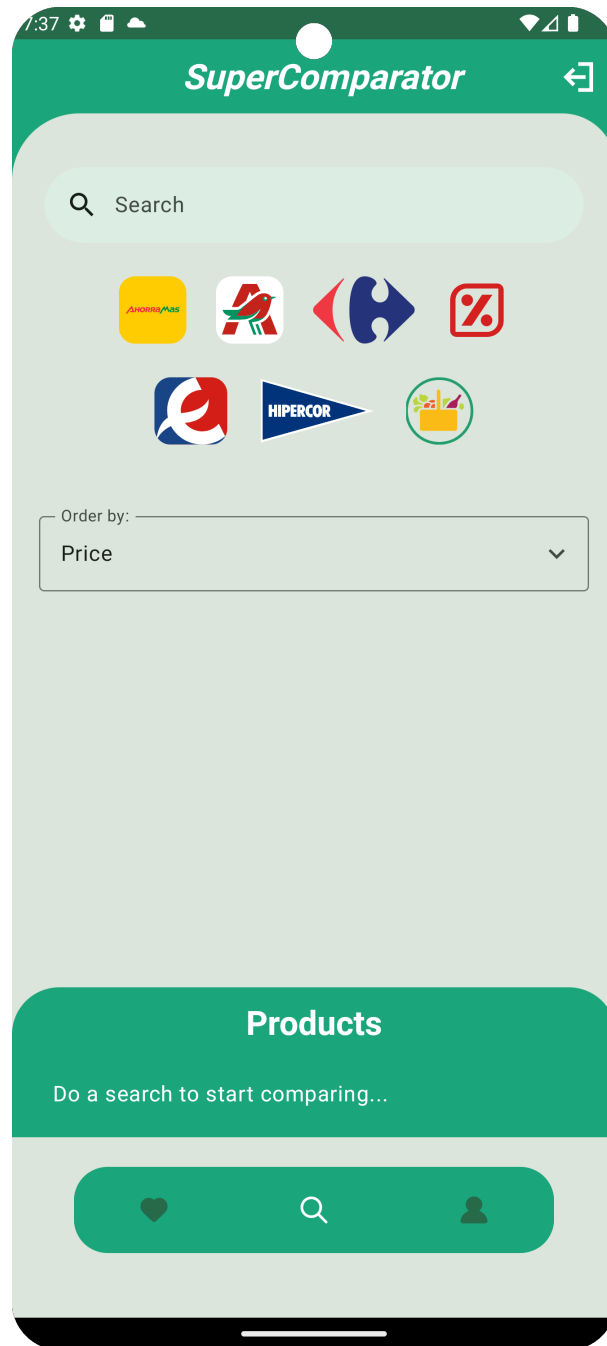


Figura 4.14: Vista: Búsqueda

Al realizar una búsqueda, la barra de búsqueda mostrará un historial reciente de búsquedas y los productos correspondientes a la consulta actual. Si se desea aplicar diferentes filtros, los componentes de los filtros actuales se eliminarán y será necesario cambiar de vista y volver para reiniciar la búsqueda.

Los resultados de la búsqueda mostrarán los productos con su precio, precio por unidad, nombre, logo de la tienda a la que pertenecen, imagen del producto y sus ofertas, si las hubiera, además de un botón de favoritos.

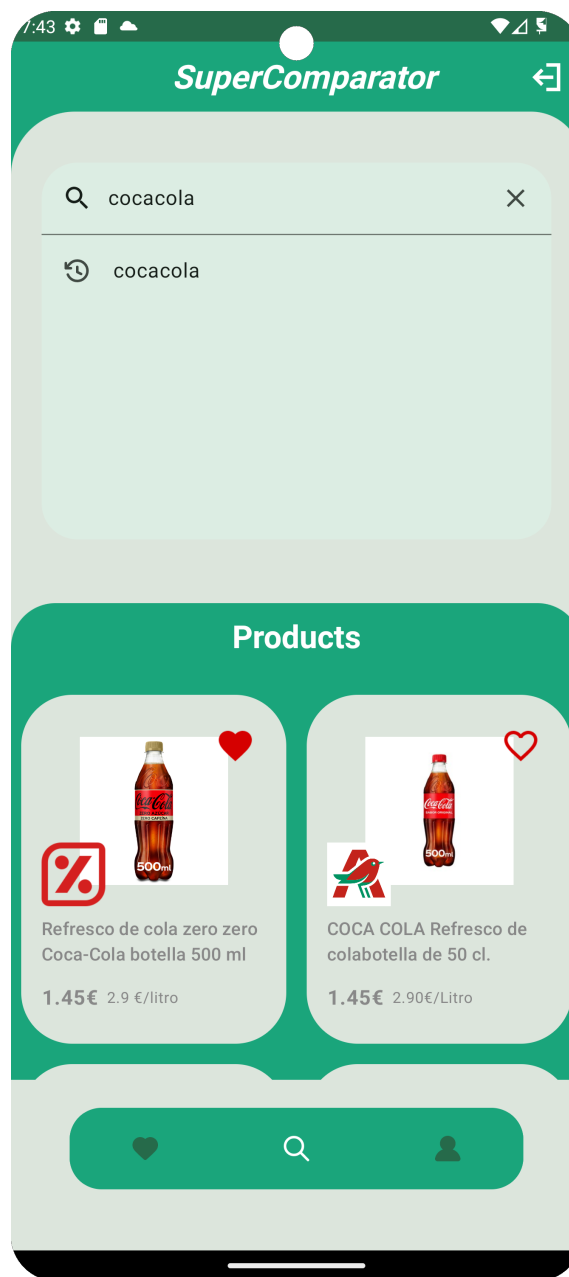


Figura 4.15: Vista: Búsqueda

En la vista de favoritos, se muestran los productos que hemos añadido a esta lista. Podemos eliminar un producto de favoritos seleccionando su icono de favoritos. Para actualizar la lista y ver los cambios, podemos cambiar de vista o refrescar la vista actual.



Figura 4.16: Vista: Favoritos

Al pulsar sobre un producto, ya sea en la búsqueda de productos o en la sección de favoritos, seremos redirigidos a una página dedicada al producto. En esta página, podremos acceder a una información más detallada, presentada de manera más amplia y legible.

Si estamos conectados a la API, tendremos la posibilidad de visualizar un gráfico que mostrará un historial de precios del producto. Además, podremos observar el precio medio a lo largo del tiempo, representado por una línea que atraviesa el gráfico.

Los datos mostrados en el gráfico de la imagen son ilustrativos, ya que no se tiene ningún cambio de ningún producto actualmente y solo se vería el precio actual.



Figura 4.17: Vista: Producto Seleccionado

Podemos cerrar sesión pulsando el icono en la parte superior derecha. Al hacerlo, aparecerá una advertencia. Si confirmamos, seremos redirigidos a la vista de inicio de sesión.

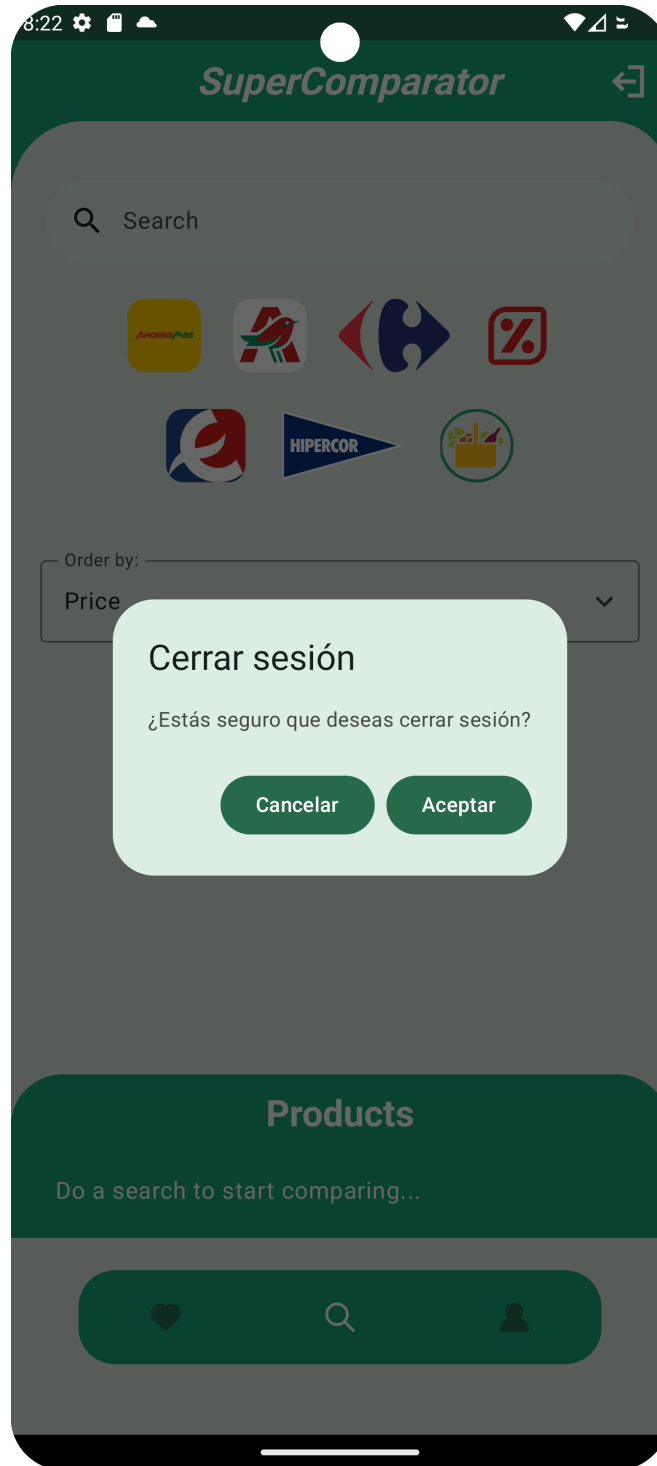


Figura 4.18: Vista: Cerrar Sesión

5

Resultados y discusión

En este capítulo, examinaremos los resultados del desarrollo de la aplicación, destacando los problemas que se han resuelto. Discutiremos las limitaciones identificadas y evaluaremos si es posible resolverlas.

El proyecto enfrentó diversos desafíos durante su desarrollo, principalmente debido a la decisión de utilizar el framework Kotlin Multiplatform. Esta elección provocó retrasos significativos al inicio del proyecto. Además, la adopción de Compose para el desarrollo de la interfaz y la integración del nuevo sistema de librerías de android añadieron más complicaciones y contribuyeron a los retrasos en un tiempo ya ajustado.

Debido a la falta de tiempo, no fue posible desarrollar la funcionalidad de consultas con múltiples parámetros. En su lugar, se decidió enfocar los esfuerzos en las nuevas funcionalidades, asegurando que las características principales estuvieran completamente implementadas y optimizadas.

La aplicación no posee todas sus funcionalidades de manera autónoma y depende de la API para su funcionamiento completo. Esta dependencia limita su uso, pero ha permitido explorar nuevas tecnologías y metodologías. Además, ha abierto el camino hacia un formato de aplicación más escalable, orientado a un producto como servicio.

6

Conclusiones

En este capítulo, se presentarán las conclusiones derivadas de las decisiones tomadas durante el proyecto, destacando los aspectos que han tenido éxito y aquellos que no. El objetivo es comprender las buenas prácticas aprendidas y evitar errores en proyectos futuros, buscando mejorar la eficiencia en el desarrollo.

6.1. Informe post-mortem

6.1.1. Lo que ha ido bien

Implementación de Firebase

El uso de Firebase (véase la sección 4.2.6) ha significado una gran aportación para el proyecto, ya que ha facilitado la implementación principalmente de funcionalidades de inicio de sesión para los usuarios.

Gracias a Firebase, se ha implementado automáticamente el inicio de sesión con una cuenta de Google. Además, se ha facilitado el registro y autenticación de usuarios mediante correo electrónico y contraseña, permitiendo también la opción de restablecer la contraseña en caso de olvido. Asimismo, se ha añadido la posibilidad de que los usuarios inicien sesión en la aplicación de manera anónima.

Además, se ha añadido una funcionalidad de análisis que permite recopilar datos de la aplicación, como los productos más consultados, la ubicación desde

donde se ha utilizado la aplicación y la cantidad de usuarios que se han conectado o registrado a lo largo del tiempo.

Uso de buenas prácticas (clean code y arquitecturas de código)

El uso de clean code y arquitecturas como la hexagonal en la API REST (véase la sección 4.2.10) ha sido fundamental. Estas buenas prácticas han agilizado el desarrollo de la aplicación, mejorando su mantenimiento a lo largo del tiempo, aumentando su legibilidad y facilitando su comprensión. Además, han permitido ahorrar tiempo en la refactorización de código y la corrección de errores. La implementación de logs claros a lo largo del programa también ha sido crucial para el desarrollo, proporcionando una mejor visibilidad y facilitando la identificación de problemas.

Se ha adoptado un enfoque orientado a la escalabilidad del proyecto, enfocando el desarrollo en clean architecture para permitir la adición de nuevas funcionalidades con mínimas modificaciones de código. Un ejemplo destacado es la API REST, que sigue una arquitectura hexagonal. Esta arquitectura permite añadir funcionalidades de manera sencilla al incorporar la lógica en los módulos de infraestructura y conectarlos a través de las interfaces de los puertos y los casos de uso. Este enfoque garantiza que cada componente sea independiente, con bajo acoplamiento y alta cohesión.

6.1.2. Lo que ha ido mal

Elección de un framework demasiado reciente

- Para el desarrollo de la aplicación, se eligió la opción de usar Kotlin Multiplataforma. La principal motivación fue poder desarrollar aplicaciones multiplataforma, escribiendo el código una vez y ejecutándolo en Android, iOS, Escritorio e incluso Web con una única implementación.
- La experiencia resultó ser muy tediosa, ya que encontrar información sobre las librerías y cómo usarlas es complicado. Existe un repositorio que posee una recopilación de librerías compatibles con Kotlin Multiplataforma en el repositorio *Awesome Kotlin Multiplatform* [1], ya que las librerías de Kotlin regulares no son compatibles.
- La importación también es diferente. Por ejemplo, la siguiente línea muestra cómo importar una librería en Gradle (Kotlin) y en Version Catalog (Kotlin-Multiplatform):

Código 6.1: Ejemplo de importación de librerías en Gradle.

```
val voyagerVersion = "1.0.0"
```

```
implementation  
( "cafe.adriel.voyager:voyager-navigator:$voyagerVersion" )
```

Código 6.2: Ejemplo de importación de librerías en Version Catalog.

```
// libs.versions.toml  
  
voyager = "1.0.0"  
  
voyager = { group = "cafe.adriel.voyager"  
name = "voyager-navigator", version.ref = "coil" }  
  
//build.gradle.kts (Module :app)  
  
implementation(libs.voyager)
```

- La escasa documentación sobre cómo usar el framework, sumada a que la implementación de la interfaz gráfica era ligeramente distinta y tenía limitaciones en comparación con Android, llevó a abandonar la idea de desarrollar la aplicación en Kotlin Multiplataforma.

Consultas poco estandarizadas e inconvenientes

- Para el proyecto Supercomparator, se han llevado a cabo múltiples consultas mediante web scraping (véase la sección 4.2.4). Sin embargo, cada página presenta sus propias particularidades, lo que demanda métodos distintos para extraer la información deseada.

Idealmente, habría sido beneficioso que todas las páginas dispusieran de una api que proporcionara un archivo json con los datos de los productos. Esto habría simplificado considerablemente la obtención de la información y mejorado el rendimiento de la aplicación. A pesar de ello, incluso en esos casos se ha tenido que filtrar el archivo json debido a su extensión, ya que su transformación completa a una clase de Kotlin no aportaría información relevante.

En aquellos casos en los que la api no proporciona un archivo json, se nos suministra el html puro de la página, el cual debemos filtrar según sus clases de css.

- El principal inconveniente ha sido con la página web de Mercadona. En esta página, las consultas no funcionaban debido a que se necesita JavaScript para poder ver el contenido de la página. A continuación, dejo un ejemplo de lo que nos devuelve una consulta haciendo una petición GET:

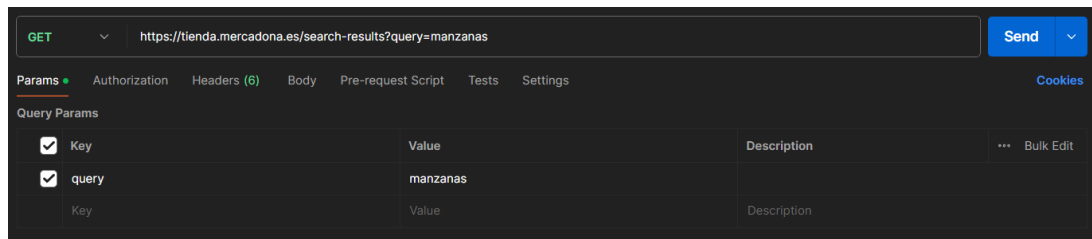


Figura 6.1: Petición a la API de Mercadona

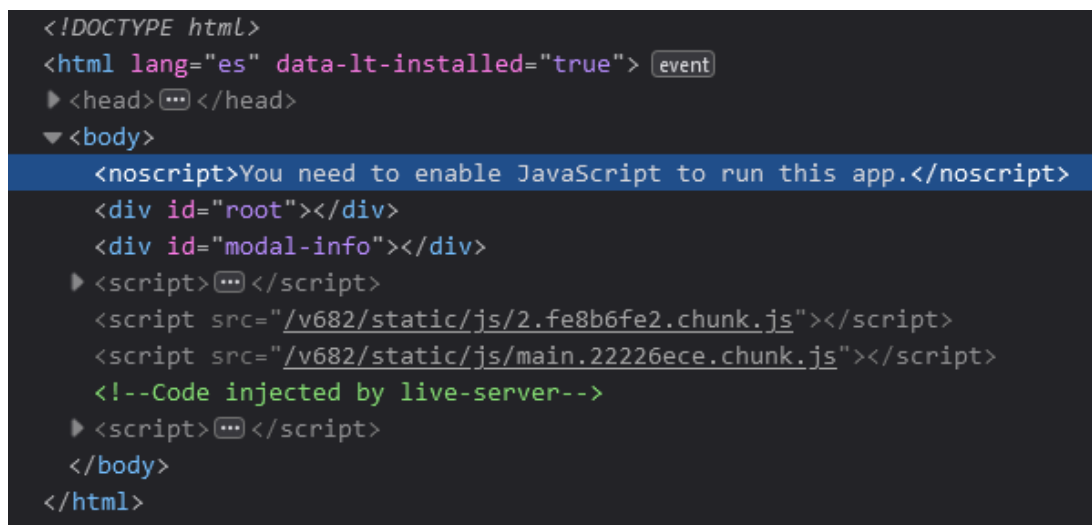


Figura 6.2: Respuesta de la petición a la API de Mercadona

- Para llevar a cabo las consultas necesarias, se ha tenido que recurrir a Selenium (véase la sección 4.2.5). Esto ha representado un gran inconveniente, ya que no se puede ejecutar la biblioteca de manera nativa en Android. La solución que se ha encontrado es asociar las búsquedas a una API REST en un servidor (véase la sección 4.2.7).

6.2. Trabajos futuros

Han quedado pendientes los objetivos de realizar consultas con múltiples parámetros y desarrollar la aplicación de manera multiplataforma. En un futuro, se podría agregar la funcionalidad de búsqueda con múltiples parámetros. Aunque adaptar la aplicación a un framework multiplataforma es posible, esto dependerá del desarrollo y las librerías disponibles.

En un futuro, sería interesante que, además de buscar la información básica de los productos, también se pudiera recopilar la información nutricional de los mismos. Las tiendas de Día, Hipercor, Alcampo y Eroski poseen información nutricional de sus productos. Se podría recopilar esta información al igual que se ha hecho con los productos de otras tiendas. En los casos en que las tiendas no posean la información, los usuarios podrían proporcionarla.

Sería interesante autenticar a los usuarios mediante un método que verifique su identidad, como el que provee Firebase. Esto permitiría que los usuarios tengan una copia de sus ajustes, historial y productos favoritos, y puedan sincronizar la información entre dispositivos mediante la base de datos de la API.

Bibliografía

- [1] Terrakok. *Awesome Kotlin Multiplatform*, 2021. [Fuente](#)
- [2] Geeksforgeeks. *What is Web Scraping and How to Use It?*, 07 Mar, 2024 [Fuente](#)
- [3] El blog python *Guía de Selenium WebDriver en Python para automatizar pruebas web* [Fuente](#)
- [4] Súper más barato. *Súper más barato* [Fuente](#)
- [5] Soysuper *Soysuper más barato* [Fuente](#)
- [6] Google *Firebase* [Fuente](#)
- [7] IBM *¿Qué es Java Spring Boot?* [Fuente](#)
- [8] Baeldung *REST with Spring Tutorial* [Fuente](#)
- [9] Android *Compila mejores apps más rápido con Jetpack Compose* [Fuente](#)
- [10] Baeldung *Organizing Layers Using Hexagonal Architecture, DDD, and Spring* [Fuente](#)
- [11] Kotlin *Kotlin Multiplatform* [Fuente](#)
- [12] Baeldun *Introduction to Retrofit* [Fuente](#)
- [13] Builtin *What Is MongoDB?* [Fuente](#)

